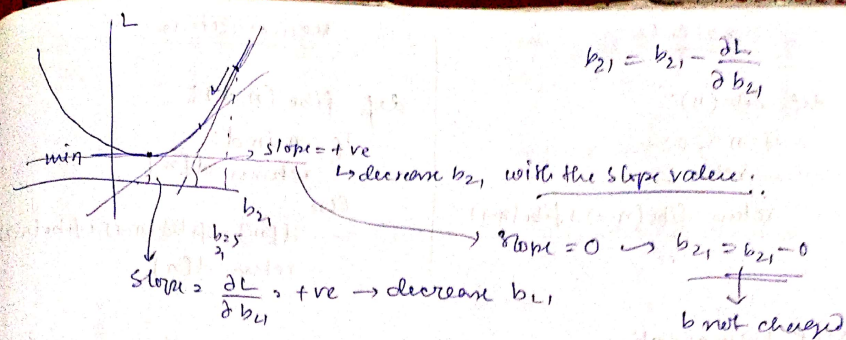


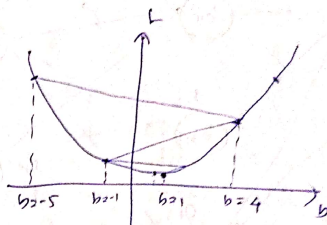


Effect of Learning Rate (η)

$$w_{\eta} = w_0 - \eta \frac{\partial L}{\partial w_0}$$

smooth

apply $\eta = 0.01$

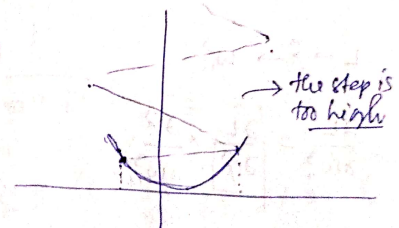


if you take the η very high - then can be a chance of never convergence

What is convergence?

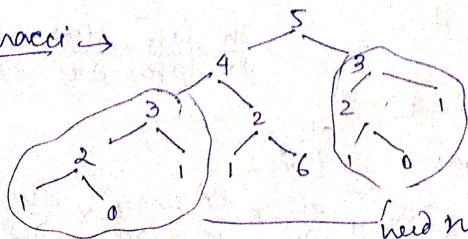
$$w_{\eta} = w_0 - \eta \frac{\partial L}{\partial w_0}$$

when $(w_{\eta} = w_0) \rightarrow$ minimum loss achieved.



Memorization $\rightarrow$ Dynamic Programming

fibonacci $\rightarrow$



need not to calculate 2 times.

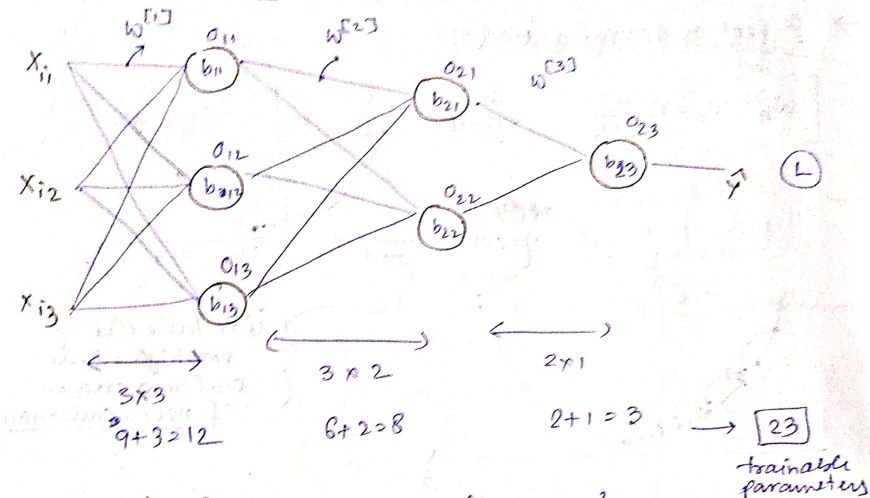
Normal code

```
def fibo(n):
    if n == 0:
        return 1
    return fibo(n-2) + fibo(n-1)
```

Memorization

```
def fibo(n, d):
    if n in d:
        return d[n]
    else:
        d[n] = fibo(n-1) + fibo(n-2)
    return d[n]
```

MLP Memorization



$$L \rightarrow \hat{y} \rightarrow w_{11}^3$$

$$L \rightarrow \hat{y} \rightarrow o_{21} \rightarrow w_{11}^2$$

$$\frac{\partial L}{\partial w_{11}^3} = \left[\frac{\partial L}{\partial \hat{y}} \right] \times \frac{\partial \hat{y}}{\partial w_{11}^3}$$

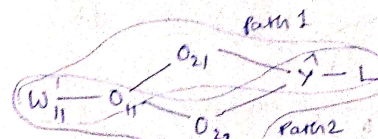
$$\frac{\partial L}{\partial w_{11}^2} = \left[\frac{\partial L}{\partial \hat{y}} \right] \times \frac{\partial \hat{y}}{\partial o_{21}} \times \frac{\partial o_{21}}{\partial w_{11}^2}$$

$$L \rightarrow \hat{y} \rightarrow o_{21} \rightarrow o_{11} \rightarrow w_{11}^1$$



$$h = f(w, g(u))$$

$$\frac{\partial h}{\partial x} = \left(\frac{\partial h}{\partial f(w)} \times \frac{\partial f(w)}{\partial x} \right) + \left(\frac{\partial h}{\partial g(u)} \times \frac{\partial g(u)}{\partial x} \right)$$



$$\frac{\partial L}{\partial w_{11}^1}$$

$$\frac{\partial L}{\partial w_{11}^1} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial o_{21}} \times \frac{\partial o_{21}}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w_{11}^1} + \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial o_{22}} \times \frac{\partial o_{22}}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w_{11}^1}$$

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial \hat{y}} \left[\frac{\partial \hat{y}}{\partial o_{21}} \times \frac{\partial o_{21}}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w_{11}} + \frac{\partial \hat{y}}{\partial o_{22}} \times \frac{\partial o_{22}}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w_{11}} \right]$$

↳ very complex with only 2 hidden layers

Backpropagation → chain diff (rule) + Memorization

↓ ↓

memos CS (op)

↳ Store the derivations while calculated one time.

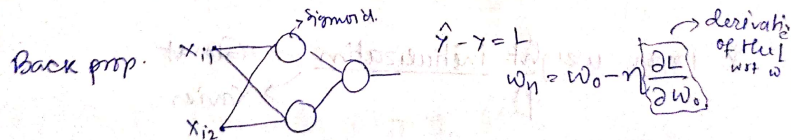
Vanishing Gradient Problem

In ML, the vanishing gradient problem is encountered when training ANNs with GD and backpropagation. In such methods, during each iteration of train each of the NN's weights receives an update proportional to the partial derivative of the error function w.r.t. the current weight. In worst case, this may stop the training of NN.

1) $0.1 \times 0.1 \times 0.1 \times 0.1 \rightarrow \boxed{0.0001} \rightarrow \text{VGP.}$

2) Deep NN → $\boxed{} \boxed{} \boxed{} \boxed{} \boxed{}$

3) Sigmoid/tanh → Activation fn →



$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w_{11}}$$

↑ ↑

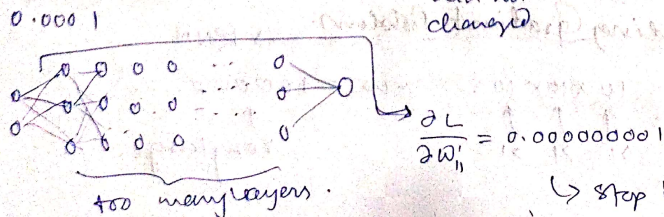
$0 < x < 1$ $0 < x < 1$

$w_{11} = 1 - 0.01 \times 0.0001$

↑

$w_{11} = 0.99999$

did not change

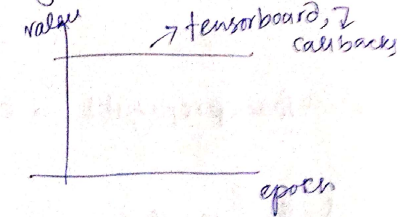


How to recognize VGP?

1) Loss focus → epoch → no changes → VGP

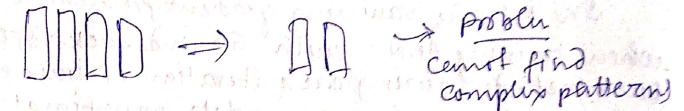
2) weights → graph

keras
↓
loss after every epoch

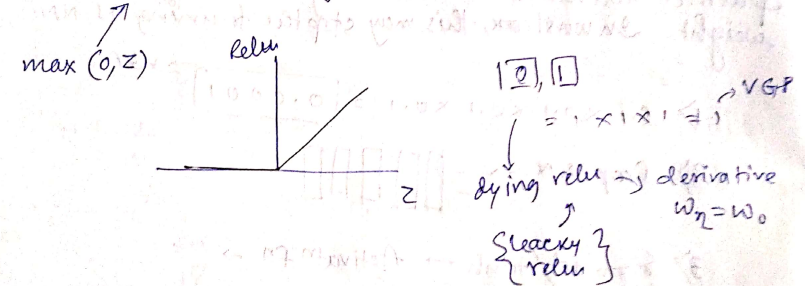


How to handle Vanishing Gradient Problem?

1) Reduce model complexity



2) Using ReLU Activation functions



3) Proper weight initialization

↑

Glorot
Xavier

4) Batch normalization → layer.

5) Residual Network → CNN (ResNET)

↳ Building Block → ANN

Exploding Gradient Problem → RNN

$w \times 10 \times 10 \dots \times w \approx 100000 \dots$

↑ ↑ ↑

> 1 > 1 > 1

very large.